This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

CVH-REC: A novel method for web API recommendation based on cross-view HGNNs

Shanquan Gao, Yihui Wang, Zhenwei Ou

Abstract—With the advancement of service computing technology, software developers tend to consume one or more web APIs, a practice that helps them avoid the behavior of reinventing the wheel. These web APIs are capable of providing services or data on the Internet; developers can use them to create feature-rich mashups. Against this backdrop, the number of web APIs across various platforms is growing rapidly, making it increasingly challenging to identify suitable ones for upcoming mashup creation. Consequently, web API recommendation has emerged as an effective means to facilitate web API discovery. We have proposed a novel web API recommendation method called R2API, which constructs the interactions between mashups and web APIs, as well as their tag usage records, into multiple homogeneous hypergraphs and then adopts HGNNs with multi-task learning to learn entity vectors for the recommendation task. While the result is encouraging, R2API's limitation is its use of simple homogeneous hypergraphs to describe entities, failing to characterize them comprehensively and accurately. To further enhance recommendation performance, this work proposes a novel cross-view HGNNs-based web API recommendation method, namely CVH-REC. First, CVH-REC models the interactions between mashups and web APIs, as well as their tag usage records, into a multi-view knowledge graph to characterize entities more comprehensively and accurately. This knowledge graph comprises a global main hypergraph and four sub-hypergraphs from local views. Second, CVH-REC adopts a contrastive learning and multi-task learning framework to drive multiple HGNNs in jointly learning entity vectors. Third, CVH-REC leverages the SBERT model to derive the semantic vector from the mashup requirement and transfers it into the vector space of the knowledge graph with an MLP. This process enables the generation of a higher-quality requirement vector. By comparing the vector of the mashup requirement with those of web APIs, CVH-REC generates a recommendation list for mashup creation. Extensive experiments on a real-world dataset demonstrate that the proposed method outperforms baseline methods.

Index Terms—Multi-view knowledge graph, cross-view HGNNs, contrastive learning, multi-task learning, web API recommendation.

1 Introduction

Given the ever-evolving business demands in the consumer market, companies must iterate rapidly during software development and shorten the development cycle. This allows them to meet consumers' business needs with maximum efficiency [1]. Against this backdrop, a growing number of software developers are turning their attention to mashup application, which is implemented by combining one or more web APIs that can provide services or data on the Internet [1], [2], [3]. With the explosive growth of web APIs, developers often struggle to efficiently identify the web APIs that meet their requirements [4], [5], [6]. As a result, web API recommendation has emerged as an effective means to facilitate web API discovery [7].

The web API recommendation methods are usually classified into two main categories: content-based methods and CF (collaborative filtering)-based methods [8]. The core logic of content-based methods is to recommend web APIs by matching requirement texts against the textual descriptions of web APIs. Nevertheless, because of the semantic gap between different data types, their performance remains unsatisfactory [9]. In contrast, CF-based methods handle mashup requirements by first identifying existing mashups with high similarity to the mashup requirement, and then generating recommended web APIs based on those adopted by these similar mashups. Despite the notable achievements of many CF-based methods, they tend to have a problem of popularity bias [10], [11]; that is, a small number of popular web APIs occupy most of the recommendation results, while most other web APIs are not well-served.

In recent years, state-of-the-art studies increasingly leverage GNNs (Graph Neural Networks) for web API recommendation (e.g., [12], [13], [14]), given their capacity to capture the high-order interactions between data. In our prior work [15], we proposed a novel web API recommendation method called R2API. This method constructs four homogeneous hypergraphs for mashups, web APIs, mashup tags, and web API tags, and then adopts four HGNNs (Hypergraph Neural Networks) with multi-task learning to learn entity vectors for the recommendation task. Benefitting from its enhanced capability in capturing high-order interactions, R2API outperforms other GNN-based methods in recommendation performance. While the result is encouraging, R2API still has a significant limitation affecting its performance: it describes mashup or web API entities with a simple homogeneous hypergraph, which fails to comprehensively and accurately characterize the entities and thus compromises the quality of the learned entity vectors.

To further assist software developers in improving the efficiency of mashup creation, this work proposes a novel cross-view HGNNs-based web API recommendation method, i.e., CVH-REC. This method involves three main

- *Shanquan Gao, Yihui Wang (corresponding author), Zhenwei Ou are with School of Artificial Intelligence(School of Software),Yanshan University; and also with Hebei Key Laboratory of Computer Virtual Technology and System Integration, Qinhuangdao, CHN.*

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

steps, as follows.

First, CVH-REC models the interactions between mashups and web APIs, along with their tag usage records, into a multi-view knowledge graph. The knowledge graph comprises two parts: a global main hypergraph and four homogeneous sub-hypergraphs. The main hypergraph includes all types of entities and records the interactions between them; the four homogeneous sub-hypergraphs, by contrast, are constructed specifically for mashups and web APIs, derived from two local views: behavioral similarity and semantic similarity. The sub-hypergraphs describe the relationships between entities of the same type, and they can form an effective complement to the main hypergraph, thereby characterizing the mashup and web API entities more comprehensively and accurately.

Second, CVH-REC adopts a contrastive learning and multi-task learning framework to enable multiple cross-view HGNNs to jointly learn entity vectors. Contrastive learning incorporates two local contrastive learning tasks and two global contrastive learning tasks. This design enhances the quality of entity vectors by enabling them to focus on core semantics, aligning semantic spaces across various views, and mitigating the issue of sparse supervision signals. Multi-task learning is performed based on the interactions between mashups and web APIs as well as their tag usage records, and it helps alleviate overfitting during model training. Additionally, by integrating knowledge across different tasks, the multi-task learning can enhance the quality of entity vectors.

Third, CVH-REC transfers the semantic vector of the mashup requirement into the vector space of the knowledge graph for web API recommendation. Specifically, CVH-REC leverages the SBERT model [16] to process the mashup requirement and derive its semantic vector, and then transfers this semantic vector into the vector space of the knowledge graph with an MLP (Multi-Layer Perceptron) [17]. The vector transfer operation enables the integration of semantic information with the structured knowledge of the knowledge graph, thereby facilitating the generation of a higher-quality requirement vector. By comparing the vector of the mashup requirement with those of web APIs, CVH-REC generates a recommendation list and presents it to software developers.

The contributions of this work are as follows:

- We characterize mashup and web API entities more comprehensively and accurately using a multi-view knowledge graph that is composed of a global main hypergraph and multiple homogeneous sub-hypergraphs from different local views.
- We design a contrastive learning and multi-task learning framework to drive the joint learning of entity vectors by multiple HGNNs.
- We propose a method to adaptively learn the vectors of mashup requirements in the vector space of the knowledge graph, which can enhance the quality of mashup requirement vectors.
- CVH-REC is evaluated through a series of experiments using a real dataset from the Programmable platform, and the results show that CVH-REC outperforms R2API and the other baselines.
- The well-documented replication package for CVH-REC and its baseline methods is provided, strengthening the re-producibility of this paper [1].

This paper is structured as follows. Section 2 introduces the problem definition and preliminary concepts. Section 3 elaborates on the technical details of our proposed method. Section 4 presents the experiments conducted to evaluate the method. Section 5 provides a detailed overview of related work, and Section 6 concludes the paper while outlining future research directions.

2 Problem Definition and Preliminaries

To better describe the web API recommendation method proposed in this paper, we present the problem definition and preliminary concepts in this section.

2.1 Problem definition

A mashup consists of a textual description, a set of web APIs employed by it, and a set of tags associated with it. A web API consists of a textual description, a set of mashups that employ it, and a set of tags associated with it. Mashup tags typically describe the functions that mashups provide to users, while web API tags describe the support web APIs offer for mashup creation or the capabilities they provide to users. Some overlap exists between the two types of tags. A mashup requirement is a textual document representing a requirement for mashup creation. Typically, it describes the functions of the mashup to be created. Based on the interactions between mashups and web APIs in historical data, as well as their tag usage records, we define web API recommendation as selecting and suggesting a suitable set of web APIs for the mashup requirement. Specifically, web APIs recommended for a given mashup requirement should have a high probability of being adopted, while minimizing the likelihood that developers will utilize web APIs not included in the recommendation.

2.2 Preliminaries

We introduce the important preliminary concepts that underpin our proposed method, including three key concepts: HGNNs, contrastive learning, and multi-task learning. These form the essential background for understanding our method.

HGNNs. Hypergraphs, in contrast to ordinary graphs, are distinguished by their ability to connect multiple entities via a single hyperedge [18], [19]. This unique property enables hypergraphs to effectively represent complex multi-dimensional relationships. HGNNs are deep learning models based on hypergraph structures. They facilitate information transmission and aggregation across multiple entities through the bidirectional message-passing mechanism. That is, each hyperedge aggregates the features of all entities it connects to form the global information of that hyperedge; subsequently, each entity aggregates information from the hyperedges it is connected to, thereby updating its own features. Compared with traditional GNNs, which can only handle pairwise relationships, HGNNs can directly model interactions between multiple entities connected by

1. https://doi.org/10.5281/zenodo.17540258

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

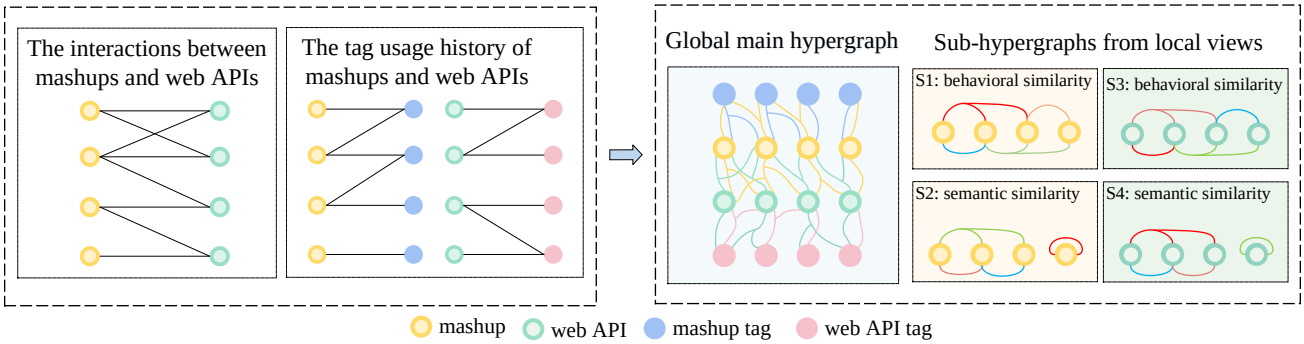


Fig. 1. The building of the multi-view knowledge graph

hyperedges, enabling more accurate mining of high-order interactions.

At layer 0, the HGNN initializes the vectors for the entities in the hypergraph. Using these, it learns entity and hyperedge vectors across all layers, with final entity vectors obtained by integrating layer-wise results. More layers enable capturing higher-order interaction information. Let $M = [m_1^*, m_2^*, \ldots, m_n^*]^T$ represent entity vectors and $E = [e_1^*, e_2^*, \ldots, e_n^*]^T$ denote hyperedge vectors; specifically, hyperedge vectors at layer l are computed via Eq.1.

$$E^{(l)} = D_e^{-1} H^T M^{(l)} \tag{1}$$

$$d(e_j) = \sum_{m_i \in H^{entity}} H(m_i, e_j) \tag{2}$$

H is the incidence matrix of the hypergraph, with rows corresponding to entities and columns to hyperedges. $H(m_i, e_j)$ represents the connection weight between entity m_i and hyperedge e_j, and 0 if unconnected. D_e is the diagonal matrix of hyperedge degrees; D_e^{-1} is used for entity information normalization. The degree of hyperedge e_j is defined in Eq.2, where H^{entity} denotes the set of entities in the hypergraph. Entities in the hypergraph can obtain their vectors at layer l+1 based on the vectors of connected hyperedges, with the calculation given by Eq.3.

$$M^{(l+1)} = D_v^{-1} H E^{(l)} \tag{3}$$

$$d(m_i) = \sum_{e_j \in H^{edge}} H(m_i, e_j) \tag{4}$$

D_v is the diagonal matrix of entity degrees, with D_v^{-1} normalizing hyperedge information. The degree of entity m_i is defined in Eq.4, where H^{edge} denotes the hyperedge set of the hypergraph. Entity vectors are derived from HGNN layer vectors, calculated as Eq.5; L is the number of embedding propagation layers; $W^{(l)}$ is the trainable weight for layer l.

$$M = \sum_{l=0}^{L} W^{(l)} M^{(l)} \tag{5}$$

Contrastive learning. The core role of contrastive learning lies in guiding models to learn more essential and robust feature representations by constructing relationships of similarity and dissimilarity between samples [20]. Contrastive learning does not rely on large amounts of labeled data; instead, it designs positive and negative sample pairs, enabling models to learn to shorten the feature distance between positive samples and lengthen that between negative samples during training. This mechanism not only helps models mine core semantics from data and enhance its anti-interference ability against noise, but also alleviates the problem of sparse supervision signals, allowing models to exhibit stronger generalization ability in downstream tasks such as classification, retrieval, and recommendation [21].

Multi-task learning. Multi-task learning is a machine learning paradigm that enables the model to learn multiple related tasks simultaneously [22], [23]. Its core lies in allowing knowledge transfer and complementation between different tasks through sharing the model structure, thereby improving the overall learning effect. The role of multi-task learning is mainly reflected in two aspects: for tasks with scarce data, it can enhance feature learning by leveraging the rich data from related tasks; at the same time, the constraints between multiple tasks can reduce the risk of overfitting and improve the model's adaptability to complex scenarios. Ultimately, multi-task learning enables the model to perform better on multiple tasks than single-task models trained independently, and is particularly suitable for scenarios where tasks are closely related.

3 Proposed Method

In this section, we present the proposed CVH-REC. CVH-REC involves three main steps: first, building the multi-view knowledge graph; second, learning entity vectors via this knowledge graph; third, recommending web APIs for specific requirements.

3.1 Building the multi-view knowledge graph

As shown in Fig.1, CVH-REC models the interactions between mashups and web APIs in historical data, along with their tag usage records, into a multi-view knowledge graph. The knowledge graph comprises a global main hypergraph and four sub-hypergraphs from local views. The main hypergraph M includes all types of entities (i.e., mashups, web APIs, mashup tags, and web API tags) and records interactions between them. It uses a hyperedge to connect each entity to those with which it interacts, with the connection weights between entities and hyperedges in the main hypergraph all set to 1.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

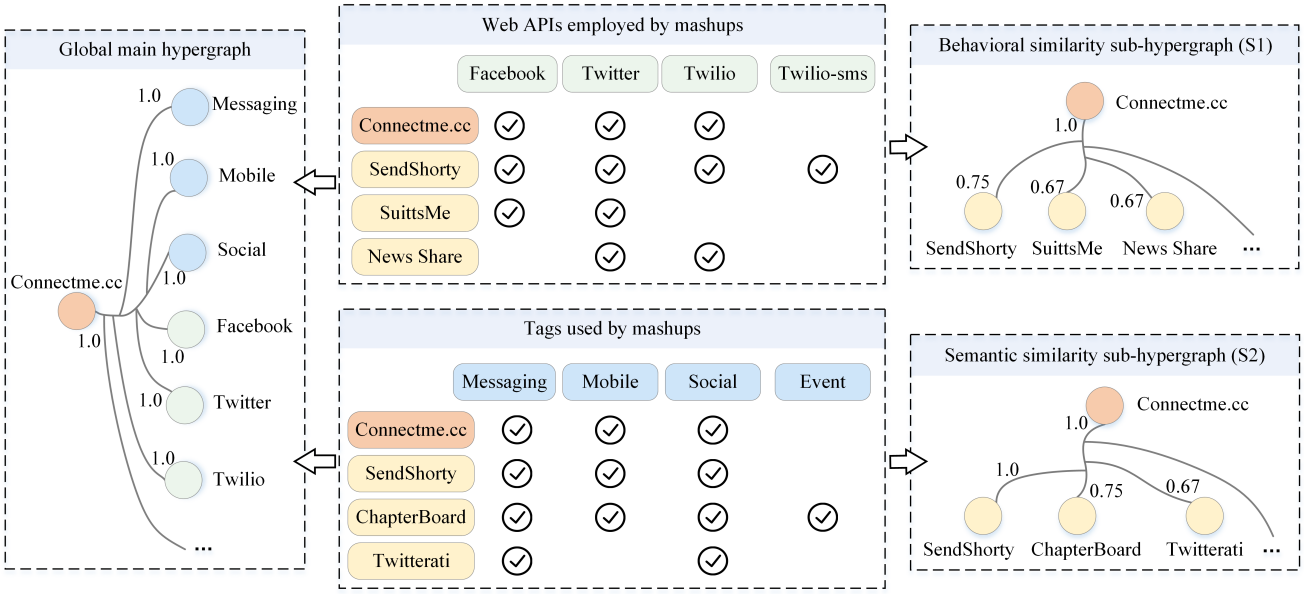


Fig. 2. The example for knowledge graph building

The four sub-hypergraphs are homogeneous hypergraphs constructed for mashups and web APIs from two local views: behavioral similarity and semantic similarity. Each entity in the sub-hypergraph is connected to its similar entities via a hyperedge. Behavioral similarity means that mashups employ similar web APIs or web APIs are employed by similar mashups; semantic similarity refers to mashups or web APIs having similar tags. Behavioral similarity reflects the practical functional correlations between entities of the same type—specifically, which web APIs are commonly employed together by mashups or which mashups share similar web API usage behaviors. This makes it a key signal for recommendation, as developers' web API selection is inherently driven by functional requirements. Meanwhile, semantic similarity supplements the correlations between entities of the same type with conceptual relevance, forming a complementary framework with behavioral similarity.

Specifically, there are the mashup behavioral similarity sub-hypergraph S1, mashup semantic similarity sub-hypergraph S2, web API behavioral similarity sub-hypergraph S3, and web API semantic similarity sub-hypergraph S4. When constructing a sub-hypergraph, CVHREC calculates the similarity scores between each entity and the other entities in the sub-hypergraph; the entities with a similarity score greater than or equal to the hyperparameter are regarded as its similar entities. The hyperparameters of sub-hypergraphs S1, S2, S3, and S4 are $\alpha 1$, $\alpha 2$, $\alpha 3$, and $\alpha 4$, respectively. The similarity scores between an entity and its similar entities are used as the connection weights between the corresponding hyperedge and these entities. The connection weight between a hyperedge and the entity that induces it is set to 1. The similarity score between two entities in S1, S2, S3, and S4 is calculated according to Eq.6, Eq.7, Eq.8, and Eq.9, respectively. $A(m_i)$ denotes the set of web APIs employed by the entity m_i; $T(m_i)$ is the set of tags associated with the entity m_i; $M(m_i)$ represents the set of mashups that employ the entity m_i.

$$Similarity_S1(m_i, m_j) = \frac{|A(m_i) \cap A(m_j)|}{|A(m_i)| + |A(m_j)| - |A(m_i) \cap A(m_j)|} \tag{6}$$

$$Similarity_S2(m_i, m_j) = \frac{|T(m_i) \cap T(m_j)|}{|T(m_i)| + |T(m_j)| - |T(m_i) \cap T(m_j)|} \tag{7}$$

$$Similarity_S3(m_i, m_j) = \frac{|M(m_i) \cap M(m_j)|}{|M(m_i)| + |M(m_j)| - |M(m_i) \cap M(m_j)|} \tag{8}$$

$$Similarity_S4(m_i, m_j) = \frac{|T(m_i) \cap T(m_j)|}{|T(m_i)| + |T(m_j)| - |T(m_i) \cap T(m_j)|} \tag{9}$$

Let us take a mashup named Connectme.cc in our experiment as an example to illustrate the building of the global main hypergraph and sub-hypergraphs from different views. As shown in Fig. 2, Connectme.cc employs three web APIs, namely Facebook, Twitter, and Twilio; meanwhile, it uses Messaging, Mobile, and Social as its tags. In the main hypergraph, Connectme.cc is connected to the mentioned web APIs and tags via a single hyperedge; the connection weights between this hyperedge and all entities are all set to 1. Since Connectme.cc is very similar to SendShorty, SuittsMe, and News Share in terms of web API usage, it is connected to these mashups via a hyperedge in the behavioral similarity sub-hypergraph; the similarity scores between Connectme.cc and mashups serve as the connection weights between the hyperedge and mashups. Similarly, since the tags of Connectme.cc are very similar to those of SendShorty, ChapterBoard, and Twitterati, it is connected to them via a hyperedge in the semantic similarity sub-hypergraph. It can be seen from this example that the entities connected to a given entity in the two sub-hypergraphs are not completely consistent.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

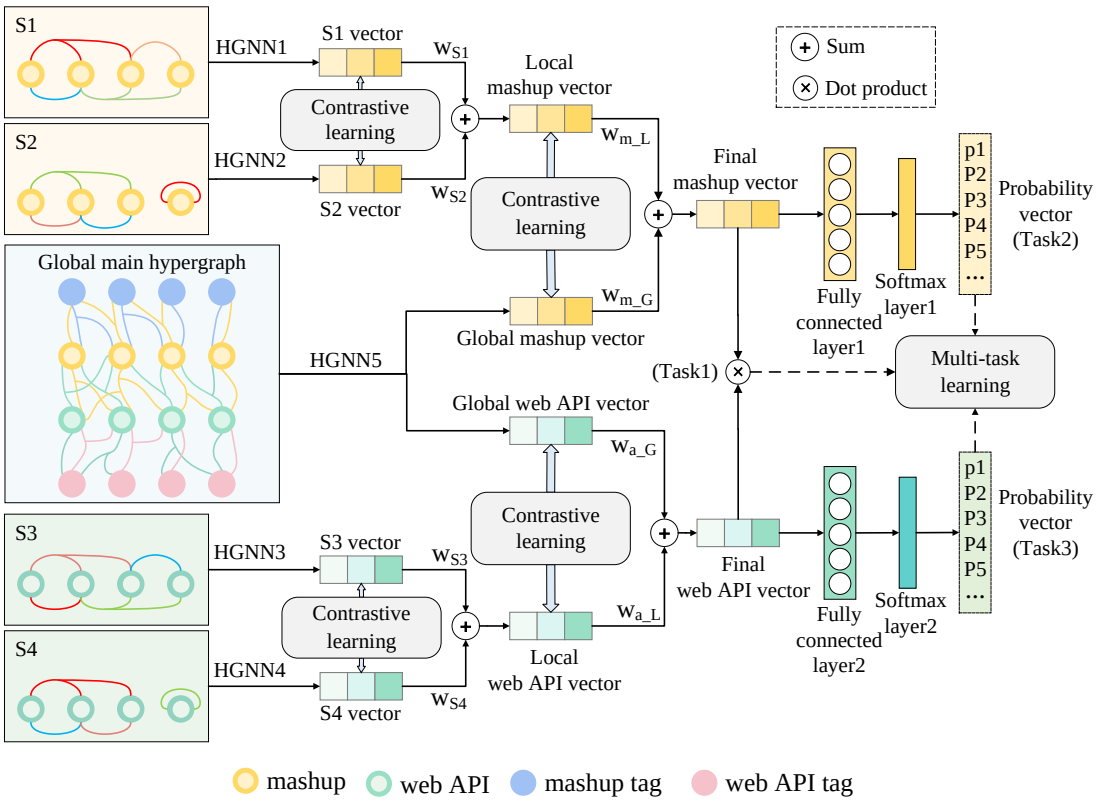


Fig. 3. The learning of the multi-view knowledge graph

3.2 Learning entity vectors via the multi-view knowledge graph

After constructing the knowledge graph, CVH-REC utilizes a dedicated HGNN for each hypergraph to learn the vectors of entities within it. As shown in Fig. 3, CVH-REC calculates the weighted average of the vectors learned by each mashup or web API from the two sub-hypergraphs as its local vector. Each sub-hypergraph is configured with a trainable weight. The final vector of each mashup or web API is the weighted average of its local vector and its global vector learned from the main hypergraph. CVH-REC configures two trainable weights for local information and global information.

During the training phase, CVH-REC leverages a contrastive learning and multi-task learning framework to drive the adjustment of model parameters. Contrastive learning can enhance the quality of entity vectors by enabling entity vectors to focus on core semantics, aligning vector spaces across views, and alleviating the issue of sparse supervision signals. The contrastive learning includes two local contrastive learning tasks and two global contrastive learning tasks: the local contrastive learning task of mashups, the local contrastive learning task of web APIs, the global contrastive learning task of mashups, and the global contrastive learning task of web APIs. Local contrastive learning tasks enhance the consistency and complementarity of information between sub-hypergraphs by aligning the vectors of the same entity in different sub-hypergraphs, thereby improving the robustness of local features; global contrastive learning tasks avoid the separation of local information from global semantics by correlating the local vectors and global vectors of entities, enabling the final vectors to incorporate both fine-grained details and overall associations.

Before completing the contrastive learning tasks, CVH-REC adopts MLPs to map the original vectors into the contrastive spaces. The MLP acts as a cross-view semantic translator in this process: it eliminates the spatial heterogeneity of the original vectors through non-linear mapping and constructs a semantically consistent contrastive space where the contrastive loss can accurately capture the semantic similarity between entities. Each contrastive learning task is configured with an MLP, and each MLP consists of three layers: an input layer, a hidden layer, and an output layer. Taking a mashup entity m_i in the local contrastive learning task of mashups as an example, its vectors learned from the two sub-hypergraphs can be mapped to the contrastive space via the MLP described in Eq.10 and Eq.11. $m_i^{(S1)}$ and $m_i^{(S2)}$ are the vectors of m_i learned from the sub-hypergraphs S1 and S2 respectively; $C_{m_i}^{(S1)}$ and $C_{m_i}^{(S2)}$ are their corresponding vectors in the contrastive space. σ is a non-linear activation function; $W^{(1)}$ and $W^{(2)}$ are trainable weight matrices; $b^{(1)}$ and $b^{(2)}$ are biases.

$$C_{m_i}^{(S1)} = W^{(2)}\sigma\left(W^{(1)}m_i^{(S1)} + b^{(1)}\right) + b^{(2)} \tag{10}$$

$$C_{m_i}^{(S2)} = W^{(2)}\sigma\left(W^{(1)}m_i^{(S2)} + b^{(1)}\right) + b^{(2)} \tag{11}$$

As shown in Fig.4(a), vectors of the same entity learned from two different sub-hypergraphs are used as a positive pair for the local contrastive learning task, whereas its vector from one sub-hypergraph and those of other entities from another form negative pairs. The local contrastive learning task aims to make positive pairs converge in the contrastive

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

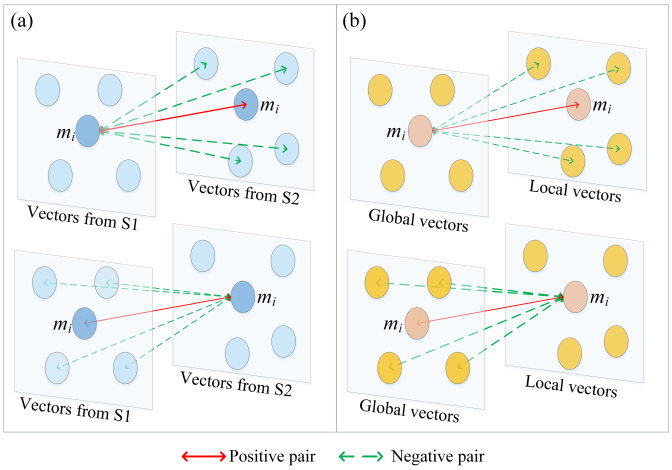


Fig. 4. The purpose of contrastive learning tasks

space while pushing negative pairs apart. Taking a mashup entity m_i as an example, its loss in the local contrastive learning task is calculated according to Eqs. 12 - 14. τ is a temperature parameter; $S(m_i)$ is the set of other entities belonging to the same type as the entity m_i.

$$Loss_{m_i}^{local} = L_{m_i}^{S1} + L_{m_i}^{S2} \tag{12}$$

$$L_{m_i}^{S1} = -\ln\left(\frac{e^{(C_{m_i}^{(S1)}\cdot C_{m_i}^{(S2)})/\tau}}{e^{(C_{m_i}^{(S1)}\cdot C_{m_i}^{(S2)})/\tau} + \sum\limits_{m_j\in S(m_i)} e^{(C_{m_i}^{(S1)}\cdot C_{m_j}^{(S2)})/\tau}}\right) \tag{13}$$

$$L_{m_i}^{S2} = -\ln\left(\frac{e^{(C_{m_i}^{(S2)}\cdot C_{m_i}^{(S1)})/\tau}}{e^{(C_{m_i}^{(S2)}\cdot C_{m_i}^{(S1)})/\tau} + \sum\limits_{m_j\in S(m_i)} e^{(C_{m_i}^{(S2)}\cdot C_{m_j}^{(S1)})/\tau}}\right) \tag{14}$$

As shown in Fig. 4(b), the global vector and local vector of an entity are used as a positive pair for the global contrastive learning task, whereas the global vector of an entity and the local vectors of other entities of the same type form negative pairs; the local vector of an entity and the global vectors of other entities of the same type also form negative pairs. Taking an entity m_i as an example, its loss in the global contrastive learning task is calculated according to Eqs. 15 - 17. $C_{m_i}^{(g)}$ is the vector in the contrastive space that corresponds to the global vector of m_i, while $C_{m_i}^{(l)}$ is the vector corresponding to its local vector.

$$Loss_{m_i}^{global} = L_{m_i}^{g} + L_{m_i}^{l} \tag{15}$$

$$L_{m_i}^{g} = -\ln\left(\frac{e^{(C_{m_i}^{(g)}\cdot C_{m_i}^{(l)})/\tau}}{e^{(C_{m_i}^{(g)}\cdot C_{m_i}^{(l)})/\tau} + \sum\limits_{m_j\in S(m_i)} e^{(C_{m_i}^{(g)}\cdot C_{m_j}^{(l)})/\tau}}\right) \tag{16}$$

$$L_{m_i}^{l} = -\ln\left(\frac{e^{(C_{m_i}^{(l)}\cdot C_{m_i}^{(g)})/\tau}}{e^{(C_{m_i}^{(l)}\cdot C_{m_i}^{(g)})/\tau} + \sum\limits_{m_j\in S(m_i)} e^{(C_{m_i}^{(l)}\cdot C_{m_j}^{(g)})/\tau}}\right) \tag{17}$$

The multi-task learning can help alleviate overfitting in the model training. Additionally, by integrating knowledge across different tasks, it can enhance the quality of entity vectors. For these reasons, the multi-task learning is employed in the training phase of CVH-REC. The multi-task learning is implemented based on the interactions between mashups and web APIs, as well as their tag usage records. Specifically, the multi-task learning includes three tasks: Task 1 aims to make the vectors of interacting mashups and web APIs approach each other; Task 2 focuses on predicting the tags used by mashups; Task 3 focuses on predicting the tags used by web APIs.

CVH-REC uses the BPR (Bayesian Personalized Ranking) to measure the loss of Task 1. Furthermore, CVH-REC inputs the vector of each mashup or web API into a predictor composed of a fully connected layer and a softmax layer to generate a probability vector. The dimension of the probability vector matches the number of tags, with each element corresponding to a tag and representing the probability of that tag being used. CVH-REC employs BCE (Binary Cross Entropy) loss to measure the discrepancy between the probability vectors and ground-truth vectors in Task 2 and Task 3. Each element of the ground-truth vector corresponds to a tag, with a value of 1 if the mashup or web API uses the tag, and 0 otherwise. The loss of multi-task learning is calculated as follows. K^{mashup} is the set of mashups, and K^{API} the set of web APIs, in the knowledge graph. m_i^* is the final vector of the entity m_i. $P(m_i)$ is the probability vector of m_i; $G(m_i)$ is its ground-truth vector; $BCE(P(m_i), G(m_i))$ is the BCE loss between the two vectors.

$$Loss_{multiTask} = L_{Task1} + L_{Task2} + L_{Task3} \tag{18}$$

$$L_{Task1} = \frac{1}{|K^{mashup}|}\sum_{m_i\in K^{mashup}}\sum_{m_j\in A(m_i)}\sum_{m_k\notin A(m_i)} -\ln\sigma\left((m_i^*\cdot m_j^*) - (m_i^*\cdot m_k^*)\right) \tag{19}$$

$$L_{Task2} = \frac{1}{|K^{mashup}|}\sum_{m_i\in K^{mashup}} BCE(P(m_i), G(m_i)) \tag{20}$$

$$L_{Task3} = \frac{1}{|K^{API}|}\sum_{m_i\in K^{API}} BCE(P(m_i), G(m_i)) \tag{21}$$

Considering the losses from both contrastive learning and multi-task learning, the objective function of model training is given by Eq.22. β is a hyperparameter that controls the impact of contrastive learning on model training. $||\theta||^2$ is the L2 regularization used to mitigate the overfitting problem; θ denotes all trainable parameters of CVH-REC. The hyperparameter λ controls the L2 regularization strength. The settings of all hyperparameters in CVH-REC

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

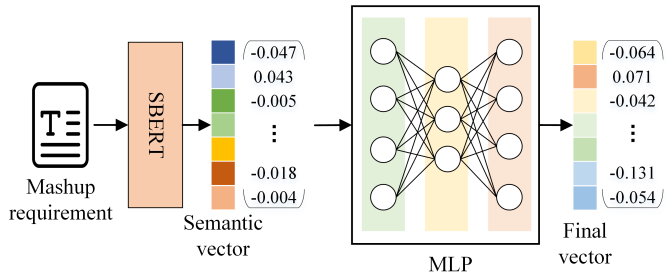


Fig. 5. The vector generation of the mashup requirement

are selected based on the validation set, that is, those that perform best on it.

$$Loss_{model} = Loss_{multiTask} + \beta \cdot (\frac{1}{|K^{mashup}|} \sum_{m_i \in K^{mashup}} (Loss_{m_i}^{global} + Loss_{m_i}^{local}) + \frac{1}{|K^{API}|} \sum_{m_j \in K^{API}} (Loss_{m_j}^{global} + Loss_{m_j}^{local})) + \lambda ||\theta||^2 \quad (22)$$

Similar to our previous work [15], HGNNs in CVH-REC do not obtain the initial entity vectors based on random vectors. Instead, CVH-REC assigns a semantic vector to each entity, and HGNNs adjust these semantic vectors to obtain the initial entity vectors during model training. The semantic vectors of entities inherently encode the domain-specific knowledge, and assigning them to entities enables HGNNs to leverage this prior knowledge during message passing. Benefiting from this point, the learned final entity vectors not only capture the structured knowledge of the knowledge graph, but also retain the domain-specific knowledge. Specifically, for each mashup or web API, CVH-REC uses SBERT to generate a vector for each sentence in its textual description, then takes the average of these vectors as its semantic vector. For each tag entity, CVH-REC directly generates a vector for the tag itself using SBERT, which serves as its semantic vector. Since the initial vectors of entities are derived from the semantic vectors output by SBERT, the final vectors of entities obtained by HGNNs have the same dimension as these semantic vectors, i.e., 768-dimensional.

3.3 Recommending web APIs for specific requirements

When dealing with a specific mashup requirement, CVH-REC leverages SBERT to process it, obtaining its semantic vector. However, instead of directly using this semantic vector as the final vector of the mashup requirement, CVH-REC transfers it into the vector space of the knowledge graph. This transfer integrates semantic information with the knowledge graph's structured knowledge, thereby facilitating the learning of a more comprehensive and precise requirement vector.

Since the dimension of the entity vectors learned from the knowledge graph is equal to that of the vectors output by SBERT, the vector dimension of the mashup requirements remains the same before and after the mapping, both being 768-dimensional. Thus, CVH-REC adapts the mashup requirement vectors to the semantic space of the knowledge graph by adjusting the vector values. As shown in Fig.5, CVH-REC employs a simple MLP to achieve the vector transfer. Specifically, after the SBERT model, an MLP is added. The MLP consists of three fully connected layers, with the dimensions of the three layers being 768, 512, and 768 respectively. The specific calculation formula for the vector transfer of the mashup requirement is defined as Eq.23. Here, r^s is the semantic vector of the mashup requirement, and r^* represents its final vector.

$$r^* = W^{(2)}\sigma\left(W^{(1)}r^s + b^{(1)}\right) + b^{(2)} \quad (23)$$

During the training phase of the MLP, CVH-REC inputs the textual descriptions of mashups in the knowledge graph into the SBERT model. By adjusting the parameters of the MLP, CVH-REC ensures the MLP's outputs converge to the mashup vectors learned from the knowledge graph. As shown in Eq.24, CVH-REC measures the loss of the MLP by calculating the MSE (Mean Squared Error) between the vectors output by the MLP and the mashup vectors learned from the knowledge graph. m_i^{MLP} is the vector output by MLP for the semantic vector of m_i; m_i^* is the vector of m_i in the knowledge graph; $MSE(m_i^{MLP}, m_i^*)$ is the mean squared error between m_i^{MLP} and m_i^*.

$$Loss_{MLP} = \frac{1}{|K^{mashup}|} \sum_{m_i \in K^{mashup}} MSE(m_i^{MLP}, m_i^*) \quad (24)$$

After learning the vector of the mashup requirement via the knowledge graph, CVH-REC evaluates the matching score between the requirement and each web API in the knowledge graph by comparing their vectors. CVH-REC then uses these high-scoring web APIs to generate a web API recommendation list. The matching score between the mashup requirement r and a web API m_i is calculated according to Eq.25, where m_i^* is the final vector of m_i.

$$MatchScore(r, m_i) = \sigma(r^* \cdot m_i^*) \quad (25)$$

4 Experiment

To evaluate the effectiveness of CVH-REC, we need to answer the following research questions:

- RQ1: How does CVH-REC perform on the web API recommendation?
- RQ2: Does the construction of sub-hypergraphs, behavioral similarity sub-hypergraphs, and semantic similarity sub-hypergraphs positively impact the performance of CVH-REC?
- RQ3: Can contrastive learning, global contrastive learning, and local contrastive learning improve the performance of CVH-REC?
- RQ4: Does multi-task learning make a positive contribution to the performance of CVH-REC?
- RQ5: How does the requirement vector transfer affect the performance of CVH-REC?
- RQ6: Can assigning the semantic vectors to entities improve the performance of CVH-REC?

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

TABLE 1
Experimental dataset

Statistics	Value
# web APIs	1,382
# mashups	7,163
# web API Invocations by mashups	13,778
# web APIs per mashup	1.92
# Words in description per mashup	27.18
# Words in description per web API	71.38

TABLE 2
Experimental setup details.

Parameter	Setting
Mashup behavioral similarity threshold ($\alpha 1$)	0.6
Mashup semantic similarity threshold ($\alpha 2$)	0.6
Web API behavioral similarity threshold ($\alpha 3$)	0.6
Web API semantic similarity threshold ($\alpha 4$)	0.6
Weight of contrastive learning (β)	0.1
Weight decay (λ)	1×10^{-4}
Optimizer	Adam
Inital learning rate	0.002
Learning rate schedule	Exponential decay schedule
Batch size	512
Epochs	200
Early stopping	Loss no improvement for 5 epochs
Hardware	NVIDIA RTX4090 24GB, with an 8124M Processor CPU*2
Framework	PyTorch 2.7, CUDA 12.6

4.1 Dataset and Experiment Setup

Our experiments used an open-source dataset crawled from ProgrammableWeb by Liu et al. [24], which contains 7,947 mashups, 1,693 web APIs, and 16,200 interaction relationships. We then performed data cleaning to filter out mashup and web API entities lacking metadata (e.g., textual descriptions, tags, and mashup-web API interaction records), yielding the final experimental dataset (see Table 1).

We used ten-fold cross-validation to evaluate the performance of CVH-REC. First, we randomly divided the mashups in the experimental dataset into a validation set and a main dataset, comprising 10% and 90%, respectively. Second, we randomly split the main dataset equally into ten sets: nine served as historical data, and the remaining one as the test set. This process was repeated ten times to ensure each set was used as the test set exactly once. Third, we trained the parameters in CVH-REC using historical data. The hyperparameters were selected based on the validation set, specifically choosing the settings that performed best on it. The specific selections of key hyperparameters and model details are shown in Table 2. Finally, we treated the textual descriptions of mashups in the test set as mashup requirements and used CVH-REC to generate a list of web APIs for each requirement. By comparing the recommended web APIs with those actually used by the mashups, we evaluated the performance of CVH-REC. In addition, details about the testbed are also presented in Table 2.

4.2 Metrics

There are numerous evaluation metrics for Recommender Systems (RS), and we have opted to utilize four commonly utilized metrics. i.e., Precision, Recall, MAP, and NDCG.

Precision. Precision measures the proportion of correctly recommended web APIs among the top-N recommended results. It is calculated according to Eq.26.

$$Precision@N = \frac{|\text{Top-N APIs} \cap \text{Ground-truth APIs}|}{|\text{Top-N APIs}|} \quad (26)$$

Recall. Recall refers to the proportion of web APIs actually invoked by the mashup that are successfully included in the top-N recommended results. It is calculated according to Eq.27.

$$Recall@N = \frac{|\text{Top-N APIs} \cap \text{Ground-truth APIs}|}{|\text{Ground-truth APIs}|} \quad (27)$$

Mean Average Precision (MAP). MAP is a ranking-sensitive metric that evaluates the quality of a ranked list by measuring how well relevant web APIs are placed at the top positions. It is defined as the mean of Average Precision (AP) over all recommendation lists in the test set. AP is calculated according to Eq.28.

$$AP@N = \frac{1}{\sum_{i=1}^{N} rel(i)} \sum_{i=1}^{N} \frac{\sum_{j=1}^{i} rel(i)}{i} \times rel(i) \quad (28)$$

where $rel(i)$ indicates whether the web API at position i is a correct one. It returns 1 if yes and 0 otherwise. $MAP@N$ is the mean $AP@N$ across all recommendation lists for requirements in the test set.

Normalized Discounted Cumulative Gain (NDCG). NDCG is a widely used metric for evaluating the ranking quality of a recommendation system. It is calculated according to Eq.29 - 31.

$$DCG@N = \sum_{i=1}^{N} \frac{rel(i)}{log_2(i+1)} \quad (29)$$

$$IDCG@N = \sum_{i=1}^{N} \frac{rel^*(i)}{log_2(i+1)} \quad (30)$$

$$NDCG@N = \frac{DCG@N}{IDCG@N} \quad (31)$$

where $rel^*(i)$ represents the relevance score at position i in an ideal ranking where all truly relevant items are placed at the top.

4.3 Baselines

To comprehensively evaluate the performance of CVH-REC, we selected several web API recommendation methods recently published in high-impact journals and conferences as the baselines, including: MTFM[2022] [8], PAREI[2023] [1], SRCLML[2024] [25], BRW[2024] [4], GSL-Mash[2024] [26], GSR[2024]

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

TABLE 3
The input provided for DeepSeek-V3

Part	Instantiation
List of candidate web APIs	The list of candidate web APIs is [idescat-population-search, google-maps, ...].
Mashup requirement	I want to complete a mashup, and the requirement is as follow: [*specific requirement*].
Task definition	Recommend 20 suitable web APIs according to my requirement.
Output format	Output the recommended web API as a python list (e.g., [API_1, API_2, ..., API_20]). Do not output anything other than the recommended web APIs.

[27], and R2API[2025] [15]. As discussed in [28], hyperparameters can play a significant role in model performance. To ensure a fair comparison, the hyperparameter settings of the baselines were kept consistent with their original papers, as we believe the settings in the original papers are optimal. Additionally, we adopted DeepSeek-V3 [29], a recently popular large language model, as a baseline for CVH-REC. To guide DeepSeek-V3 in performing the task of web API recommendation, we combine the instantiations in Table 3 as its input, i.e., <List of candidate web APIs>+<Mashup requirement>+<Task definition>+<Output format>. The list of candidate web APIs consists of the web APIs used by mashups in our experiment. The [*specific requirement*] part needs to be replaced with the requirement text of the mashup to be implemented. Moreover, the temperature of DeepSeek-V3 is set to 0.1 to reduce randomness, and thus to ensure output stability and to support experimental reproducibility.

4.4 RQ1: Performance of CVH-REC

Table 4 presents the experimental results of CVH-REC and all baseline methods. In terms of Precision, Recall, MAP, and NDCG, CVH-REC outperforms other web API recommendation methods. Among all methods, the one with performance closest to CVH-REC is R2API, a method we proposed earlier. However, as shown in Table 5, the p-values for the differences between CVH-REC and R2API in all evaluation metrics—computed using the Wilcoxon signed-rank test—are all less than $\alpha_{corrected} = 0.003125$, i.e., the Bonferroni-corrected significance level calculated by dividing $\alpha = 0.05$ (the initial significance level [30]) by $k = 16$ (the number of multiple comparisons). This indicates that the improvement in recommendation performance achieved by CVH-REC is statistically significant. Meanwhile, the Cohen's d values between the two are all greater than 0.8, which highlights that this performance improvement has important practical significance for the web API recommendation task. Our analysis identifies three reasons for CVH-Re's superior recommendation performance over R2API. First, the multi-view knowledge graph constructed by CVH-REC can characterize the mashup and web API entities more accurately and comprehensively. Second, the contrastive learning and multi-task learning framework helps CVH-REC adjust model parameters effectively. Third, CVH-Rec transfers the semantic vectors of mashup requirements into the vector space of the knowledge graph with an MLP. As a learnable non-linear mapping function, the MLP can adapt to different requirements through data-driven learning and achieve flexible vector transfer. Furthermore, it is important to note that the Recall results of many baselines in this experiment are lower than those reported in R2API's original paper. This is because the average number of web APIs used per mashup here is higher than that in R2API's original paper (1.92 versus 1.61), which increases the difficulty for the model to achieve favorable results on the Recall metric. Notably, the Recall of MTFM in this experiment is improved

TABLE 4
The evaluation results of web API recommendation methods (10-fold cross-validation, mean ± standard deviation)

Top-N	Methods	Precision	Recall	MAP	NDCG
3	MTFM[2022]	0.2856±0.0061	0.5503±0.0161	0.6505±0.0174	0.5675±0.0145
	PAREI[2023]	0.2888±0.0068	0.5501±0.0197	0.6233±0.0153	0.5523±0.0166
	SRCLML[2024]	0.2620±0.0075	0.4994±0.0167	0.6006±0.0159	0.5199±0.0150
	BRW[2024]	0.2900±0.0101	0.4688±0.0211	0.5872±0.0177	0.5067±0.0179
	GSL-Mash[2024]	0.2549±0.0074	0.4978±0.0184	0.5851±0.0197	0.5063±0.0180
	GSR[2024]	0.2819±0.0065	0.5646±0.0093	0.6719±0.0118	0.5803±0.0094
	DeepSeek-V3[2024]	0.2181±0.0080	0.4127±0.0152	0.4277±0.0140	0.3891±0.0141
	R2API[2025]	0.3123±0.0081	0.6113±0.0168	0.7123±0.0137	0.6270±0.0134
	CVH-Rec	**0.3361±0.0072**	**0.6435±0.0115**	**0.7392±0.0115**	**0.6596±0.0111**
5	MTFM[2022]	0.1907±0.0051	0.5929±0.0161	0.6517±0.0172	0.5775±0.0142
	PAREI[2023]	0.2002±0.0041	0.6134±0.0184	0.6269±0.0151	0.5719±0.0157
	SRCLML[2024]	0.1835±0.0057	0.5668±0.0137	0.6099±0.0161	0.5428±0.0133
	BRW[2024]	0.1983±0.0065	0.5092±0.0218	0.5839±0.0167	0.5111±0.0180
	GSL-Mash[2024]	0.1741±0.0060	0.5460±0.0183	0.5892±0.0180	0.5206±0.0180
	GSR[2024]	0.1880±0.0038	0.6079±0.0103	0.6720±0.0103	0.5917±0.0102
	DeepSeek-V3[2024]	0.1490±0.0054	0.4535±0.0133	0.4320±0.0150	0.4093±0.0153
	R2API[2025]	0.2103±0.0049	0.6614±0.0126	0.7124±0.0117	0.6404±0.0112
	CVH-Rec	**0.2276±0.0039**	**0.6984±0.0133**	**0.7366±0.0106**	**0.6744±0.0110**
10	MTFM[2022]	0.1092±0.0030	0.6541±0.0187	0.6408±0.0149	0.5982±0.0158
	PAREI[2023]	0.1184±0.0028	0.6967±0.0166	0.6182±0.0148	0.6010±0.0142
	SRCLML[2024]	0.1073±0.0034	0.6382±0.0120	0.6018±0.0127	0.5672±0.0123
	BRW[2024]	0.1118±0.0031	0.5571±0.0149	0.5781±0.0166	0.5241±0.0150
	GSL-Mash[2024]	0.1011±0.0033	0.6096±0.0192	0.5826±0.0194	0.5425±0.0170
	GSR[2024]	0.1070±0.0023	0.6656±0.0106	0.6608±0.0111	0.6116±0.0100
	DeepSeek-V3[2024]	0.0877±0.0050	0.5254±0.0121	0.4242±0.0093	0.4616±0.0172
	R2API[2025]	0.1208±0.0028	0.7218±0.0097	0.6991±0.0106	0.6618±0.0103
	CVH-Rec	**0.1294±0.0031**	**0.7605±0.0105**	**0.7238±0.0088**	**0.6958±0.0105**
20	MTFM[2022]	0.0616±0.0021	0.7119±0.0154	0.6250±0.0158	0.6162±0.0146
	PAREI[2023]	0.0674±0.0017	0.7626±0.0136	0.6012±0.0146	0.6216±0.0133
	SRCLML[2024]	0.0616±0.0017	0.7059±0.0087	0.5851±0.0141	0.5882±0.0112
	BRW[2024]	0.0613±0.0022	0.6074±0.0201	0.5682±0.0167	0.5384±0.0162
	GSL-Mash[2024]	0.0580±0.0020	0.6734±0.0156	0.5689±0.0185	0.5621±0.0158
	GSR[2024]	0.0605±0.0017	0.7215±0.0139	0.6426±0.0111	0.6293±0.0099
	DeepSeek-V3[2024]	0.0471±0.0019	0.5570±0.0092	0.4139±0.0136	0.5189±0.0114
	R2API[2025]	0.0681±0.0018	0.7781±0.0116	0.6801±0.0095	0.6800±0.0106
	CVH-Rec	**0.0723±0.0018**	**0.8171±0.0105**	**0.7034±0.0103**	**0.7139±0.0092**

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

TABLE 5
The p-values and Cohen's d values between CVH-REC and R2API

Metrics	Top-N	Precision	Recall	MAP	NDCG
p-value	3	0.001953	0.001953	0.001953	0.001953
	5	0.001953	0.001953	0.001953	0.001953
	10	0.001953	0.001953	0.001953	0.001953
	20	0.001953	0.001953	0.001953	0.001953
Cohen's d value	3	2.3105	1.8319	1.6504	2.0084
	5	3.1862	2.2060	1.5084	2.2595
	10	2.2779	2.7308	1.6431	2.4269
	20	2.1165	3.1135	1.5209	2.6724

TABLE 6
The training time, execution time and space complexity for each recommendation method

Methods	Training time	Execution time	Space complexity
MTFM[2022]	5-6 min	0.0005s	1113.37 MB
PAREI[2023]	8.5-9 min	0.05-0.1s	4182.42 MB
SRCLML[2024]	2-3 min	0.0005s	2523.58 MB
BRW[2024]	/	2.5-3 min	1398.36 MB
GSL-Mash[2024]	25-30 min	0.001s	3551.30 MB
GSR[2024]	8-9 min	0.0005s	7413.68 MB
DeepSeek-V3[2024]	/	12-14s	/
R2API[2025]	9-10 min	0.001s	1485.63 MB
CVH-Rec	20-25 min	0.02-0.03s	1775.69 MB

compared to the results reported in R2API's original paper. This is because the tag information of mashups in this experiment is more comprehensive—each mashup has an average of 2.46 tags in R2API's original paper, while this number is 2.93 in the current experiment. MTFM leverages the tag information of mashup requirements through a deep learning model during web API recommendation, a factor that contributes to the improvement of its Recall value. We applied CVH-REC to the dataset of R2API's original paper, and CVH-REC still maintained its advantage over R2API (when recommending 3 web APIs for each mashup requirement, CVH-REC outperforms R2API by 1.74%, 3.13%, 2.52%, and 2.68% in the four metrics of Precision, Recall, MAP, and NDCG, respectively), indicating that the advantage of CVH-REC over R2API is stable.

Table 4 also shows the experimental results of DeepSeek-V3, and the results indicate that DeepSeek-V3 performs poorly in the web API recommendation task. As discussed in [15], the subpar performance of DeepSeek-V3 lies not in a lack of web API-related knowledge, but in its extremely high demands on the quality of requirement texts. Specifically, when a mashup requirement details the functions of the mashup to be created in a professional and comprehensive manner, DeepSeek-V3 often returns reliable results. Conversely, if the requirement is unclear, its recommendations are often untrustworthy. This phenomenon indicates that DeepSeek-V3's robustness is inferior to that of specialized web API recommendation methods.

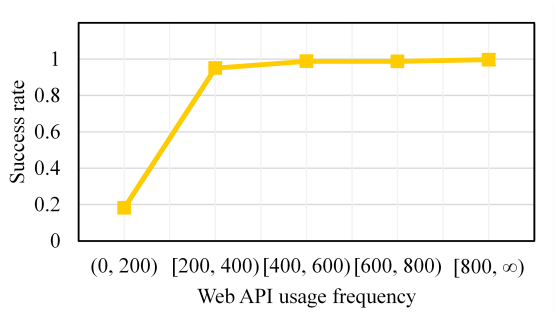


Fig. 6. The correlation between web API usage frequency and recommendation success rate

Table 6 compares the training time, execution time, and space complexity of the evaluated methods. It can be observed that CVH-REC has a relatively long training time compared to other methods. This is because it employs multiple HGNNs for the joint learning of entity vectors, while simultaneously integrating contrastive learning and multi-task learning to enhance the quality of these vectors. A larger number of training parameters directly contributes to its extended training duration. In terms of execution time, CVH-REC takes merely 0.02–0.03 s per query, highlighting its high responsiveness. In terms of space complexity, CVH-REC uses 1775.69 MB of memory, placing it at a moderate level among all evaluated methods.

Even when considering 20 web APIs, some valuable web APIs are still not recommended by CVH-REC. By examining the experimental data, we have summarized three main reasons for the failure cases of CVH-REC. First, some mashup requirements are overly abstract and fail to properly describe the functions of mashup applications, resulting in the failure of CVH-REC to provide accurate recommendations. For example, the description of the mashup named ILocy is "Love your town!". This vague statement provides little information about its specific functions. As a result, CVH-REC fails to accurately recommend the web APIs that this mashup actually relies on, namely Facebook and Instagram Graph. Second, if the web APIs required by mashup requirements are popular, CVH-REC can often recommend them correctly; however, if the required web APIs are unpopular, the probability of CVH-REC making correct recommendations decreases. In other words, CVH-REC suffers from the long-tail problem [14] to some extent. To verify this claim, we calculated the success rate of each web API. Specifically, this metric refers to the number of times a web API was correctly included in the recommendation lists (each containing 20 web APIs), divided by the total number of times it was adopted in mashups. We then categorized all web APIs into different groups based on their usage frequencies, and computed the average success rate for each group—the results are shown in Fig.6. As clearly indicated by the figure, low-frequency web APIs achieve a significantly lower success rate than high-frequency web APIs. Third, we assessed the performance of CVH-REC by comparing the web APIs it recommended against those actually employed in mashups. That said, given that mashups may allow for multiple valid combinations of web APIs to achieve the same functions, certain reasonable recommendations generated by CVH-

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

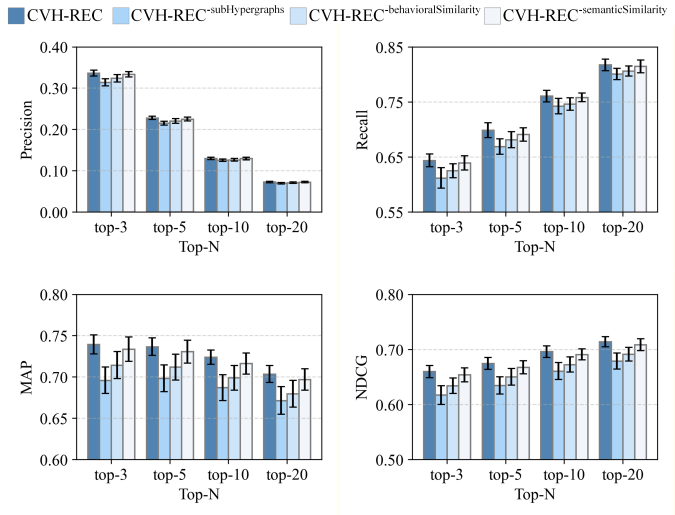


Fig. 7. Impact of sub-hypergraphs, behavioral similarity sub-hypergraphs, and semantic similarity sub-hypergraphs (10-fold cross-validation, mean ± standard deviation)

REC may have been erroneously classified as inappropriate. For instance, the mashup named Help a Classroom in Need employs two web APIs, i.e., DonorsChoose and Bing Maps, whereas the top-2 web APIs recommended by CVH-REC are DonorsChoose and Google Maps. Considering that Google Maps provides highly similar mapping functions to Bing Maps, the recommendation is reasonable despite not exactly matching those used in the mashup.

4.5 RQ2: Impact of sub-hypergraphs, behavioral similarity sub-hypergraphs, and semantic similarity sub-hypergraphs

To comprehensively and accurately characterize mashup and web API entities, CVH-REC uses the sub-hypergraphs from two local views (i.e., behavioral similarity and semantic similarity) as a complement to the global main hypergraph. In this RQ, we performed an experiment to explore the impact of sub-hypergraphs, behavioral similarity sub-hypergraphs, and semantic similarity sub-hypergraphs on the performance of CVH-REC. For this purpose, we first constructed three variants for CVH-REC, namely CVH-REC$^{\text{-subHypergraphs}}$, CVH-REC$^{\text{-behavioralSimilarity}}$, and CVH-REC$^{\text{-semanticSimilarity}}$. CVH-REC$^{\text{-subHypergraphs}}$ characterizes mashups and web APIs using only the main hypergraph, and treats vectors from the main hypergraph as entity vectors. CVH-REC$^{\text{-behavioralSimilarity}}$ uses the main hypergraph and semantic similarity sub-hypergraphs to characterize mashups and web APIs, with vectors from the semantic similarity sub-hypergraphs serving as entities' local vectors. CVH-REC$^{\text{-semanticSimilarity}}$ leverages the main hypergraph and behavioral similarity sub-hypergraphs to characterize mashups and web APIs, taking vectors from the behavioral similarity sub-hypergraphs as entities' local vectors. Then, we applied these variants to our experimental dataset and compared their performance with that of CVH-REC.

Fig.7 shows that CVH-REC$^{\text{-subHypergraphs}}$ underperforms CVH-REC, indicating that the sub-hypergraphs can help CVH-REC enhance the performance in web API recommendation. The main hypergraph records the interactions between entities of different types, while the sub-hypergraphs describe the relationships between entities of the same type from different views. The sub-hypergraphs can serve as an effective complement to the main hypergraph, enabling CVH-REC to characterize the entities more accurately and comprehensively, which is asociated with the improvement of entity vector quality. Furthermore, Fig.7 also shows that the performance of CVH-REC$^{\text{-behavioralSimilarity}}$ and CVH-REC$^{\text{-semanticSimilarity}}$ is inferior to that of CVH-REC. This indicates that both behavioral similarity sub-hypergraphs and semantic similarity sub-hypergraphs have a positive impact on the performance of CVH-REC. Behavioral similarity sub-hypergraphs and semantic similarity sub-hypergraphs describe the relationships between entities of the same type from different views, and they play a complementary role to each other. Meanwhile, the contrastive learning between these two types of sub-hypergraphs imposes constraints on the learning of entity vectors, which can help enhance the quality of entity vectors. In contrast, considering only one of the two types of sub-hypergraphs leads to the loss of such constraints from contrastive learning.

4.6 RQ3: Impact of contrastive learning, global contrastive learning, and local contrastive learning

In the learning phase of the knowledge graph, CVH-REC employs a contrastive learning and multi-task learning framework to drive multiple HGNNs in jointly learning entity vectors. In RQ3, we conducted an experiment to analyze whether contrastive learning, global contrastive learning, and local contrastive learning exert a positive impact on the performance of CVH-REC. To this end, we first constructed three variants of CVH-REC, namely CVH-REC$^{\text{-contrastive}}$, CVH-REC$^{\text{-globalContrastive}}$, and CVH-REC$^{\text{-localContrastive}}$. Unlike the variants in RQ2, which remove sub-hypergraphs, the variants in this RQ retain hypergraphs from all views; they only remove the contrastive learning tasks between different hypergraphs. Specifically, CVH-REC$^{\text{-contrastive}}$ does not consider any contrastive learning tasks; CVH-REC$^{\text{-globalContrastive}}$ does not consider global contrastive learning tasks; and CVH-REC$^{\text{-localContrastive}}$ does not consider local contrastive learning tasks. We then evaluated the impact of contrastive learning, global contrastive learning, and local contrastive learning by comparing the experimental results of variants and CVH-REC.

Fig.8 presents the experimental results for CVH-REC$^{\text{-contrastive}}$, from which it can be observed that CVH-REC$^{\text{-contrastive}}$ underperforms CVH-REC. This confirms that integrating contrastive learning enhances the recommendation performance of CVH-REC. Our analysis identifies three main reasons for this result. First, data from different views may contain noise and redundant information. Contrastive learning forces the model to learn a vector space where the vectors of the same entities are pulled closer and the vectors of different entities are pushed apart. This constraint enables entity vectors to focus more on core semantics, filtering out noise and redundant information. Second, the vector spaces of different views may exhibit semantic misalignment, and contrastive learning supervises

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

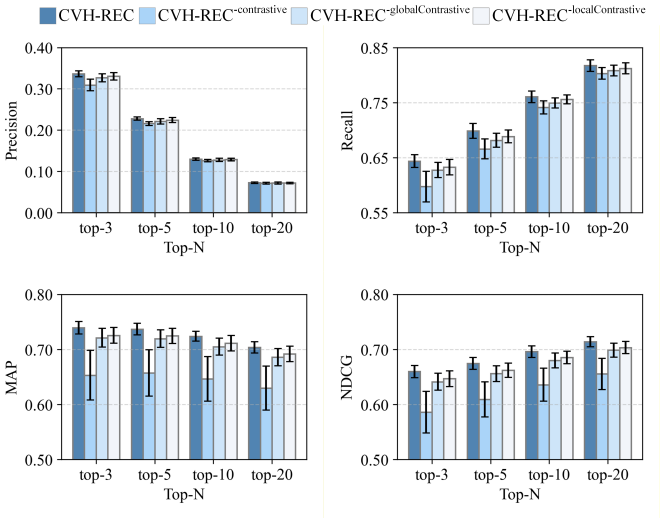


Fig. 8. Impact of contrastive learning, global contrastive learning, and local contrastive learning (10-fold cross-validation, mean ± standard deviation)

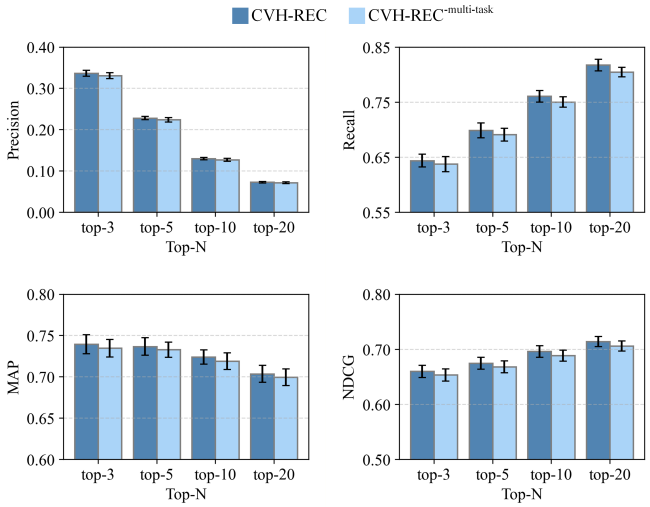


Fig. 9. Impact of multi-task learning (10-fold cross-validation, mean ± standard deviation)

the model to align the vector spaces across different views. This alignment ensures that vectors of the same entity across different views are consistent, thus avoiding the bias of a single view. Third, contrastive learning can help CVH-REC alleviate the problem of sparse supervision signals by constructing self-supervised constraints from unlabeled data, which supplements the limited explicit supervision and stabilizes the training process.

It can be seen that the performance of CVH-REC$^{\text{-globalContrastive}}$ and CVH-REC$^{\text{-localContrastive}}$ is inferior to that of CVH-REC, indicating that both global contrastive learning and local contrastive learning positively impact the recommendation performance of CVH-REC. As discussed in Section 3.2, global contrastive learning avoids the separation of local information from global semantics by correlating the local vectors and global vectors of entities; meanwhile, local contrastive learning enhances the consistency and complementarity of information between sub-hypergraphs by aligning the vectors of the same entity in different sub-hypergraphs. Furthermore, the role of global contrastive learning is greater than that of local contrastive learning, as it effectively avoids the fragmentation or noise interference of local information by forcing the alignment of local information with global information.

4.7 RQ4: Impact of multi-task learning

To enhance the quality of entity vectors, CVH-REC incorporates multi-task learning into model parameter adjustment. In this RQ, we designed an experiment to evaluate the impact of multi-task learning on the performance of CVH-REC. For this purpose, we first constructed a variant named CVH-REC$^{\text{-multi-task}}$. During the training phase, this variant only considers Task 1 while ignoring Tasks 2 and 3. We then evaluated its performance using our experimental dataset and analyzed the impact of multi-task learning by comparing the results with those of CVH-REC.

Fig.9 illustrates the experimental results of CVH-REC$^{\text{-multi-task}}$. As shown in the figure, integrating multi-task learning has a positive impact on the recommendation performance of CVH-REC. As discussed in our previous study [15], the strong information aggregation capability and representation power of HGNNs may exacerbate the adverse effects of noise and atypical features in historical data on model training, thereby negatively impacting the quality of entity vectors. Multi-task learning can help HGNNs identify more representative features from the data, enhance their ability to interpret the data, and thus mitigate the impact of noise and atypical features. Meanwhile, it can actively integrate knowledge across different tasks to further improve the quality of entity vectors.

4.8 RQ5: Impact of requirement vector transfer

CVH-REC uses an MLP to transfer the semantic vector of the mashup requirement into the vector space of the knowledge graph, so as to obtain the final requirement vector. However, since the semantic vector of the requirement already effectively captures its core features, it remains unclear whether transferring the semantic vector into the vector space of the knowledge graph can positively impact the performance of CVH-REC. Therefore, we conducted an experiment in RQ5 to explore whether the requirement vector transfer positively impacts CVH-REC. First, we constructed a variant of CVH-REC, namely CVH-REC^{semantic}. This variant uses the semantic vector of mashup requirement directly as the requirement vector. Then, we evaluated the impact of requirement vector transfer by comparing the performance of CVH-REC^{semantic} and CVH-REC.

Our previously proposed method, R2API, measures the similarity between the mashup requirement and mashups in the knowledge graph by comparing the semantic vectors, and then calculates the weighted average of vectors of top-β mashups as the requirement vector. Essentially, this operation of R2API can also be understood as transferring the semantic vector of the mashup requirement into the vector space of the knowledge graph to obtain the final requirement vector. In this RQ, we also analyzed whether the vector transfer method of CVH-REC is more reasonable

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

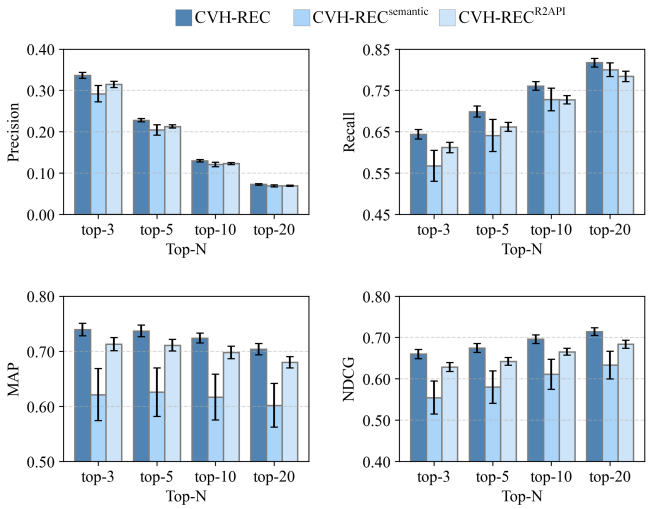


Fig. 10. Impact of requirement vector transfer (10-fold cross-validation, mean ± standard deviation)

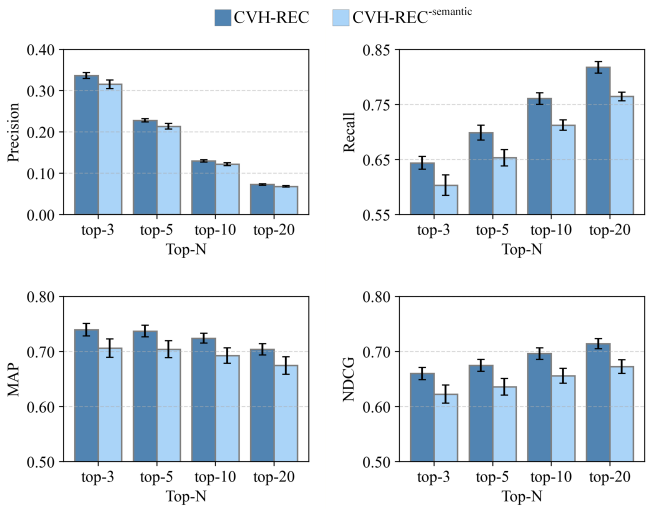


Fig. 11. Impact of assigning the semantic vectors (10-fold cross-validation, mean ± standard deviation)

than that of R2API, i.e., whether using an MLP to complete the transfer task is more advantageous. For this purpose, we constructed a variant called CVH-REC$^{\text{R2API}}$ for CVH-REC and then compared the performance of CVH-REC$^{\text{R2API}}$ with that of CVH-REC. Unlike CVH-REC, CVH-REC$^{\text{R2API}}$ obtains the requirement vector by calculating the weighted average of the vectors of top-β mashups similar to the mashup requirement.

Fig.10 presents the performance of CVH-REC^{semantic}. We can see from the figure that the experimental results of CVH-REC^{semantic} are lower than those of CVH-REC. This suggests that transferring the semantic vector of the mashup requirement into the vector space of the knowledge graph can positively affect the performance of CVH-REC. Indeed, this vector transfer enables the integration of semantic information with the structured knowledge of the knowledge graph, and thus facilitates the learning of a higher-quality requirement vector.

Furthermore, Fig.10 also shows that the performance of CVH-REC$^{\text{R2API}}$ is inferior to that of CVH-REC, which indicates that CVH-REC can complete the vector transfer task more reasonably than R2API. R2API needs to determine a value for the hyperparameter β based on the validation set. However, in reality, the optimal value of β varies with different mashup requirements, and the value of β determined from the validation set can only be considered suitable for most of the data. In contrast, CVH-REC employs an MLP to perform the vector transfer. As a learnable non-linear mapping function, the MLP can adapt to different requirements through data-driven learning and thus achieve more flexible vector transfer.

4.9 RQ6: Impact of assigning the semantic vectors

At layer 0 of HGNNs, CVH-REC assigns semantic vectors to entities in the hypergraph; HGNNs then obtain initial entity vectors by adjusting these semantic vectors. In RQ6, we conducted an experiment to evaluate whether assigning semantic vectors to entities positively contributes to the performance of CVH-REC. For this purpose, we first constructed a variant named CVH-REC$^{\text{-semantic}}$. Unlike CVH-REC, the HGNNs in CVH-REC$^{\text{-semantic}}$ derive initial entity vectors based on random vectors. We then applied CVH-REC$^{\text{-semantic}}$ to our experimental dataset and analyzed the impact of assigning semantic vectors to entities by comparing its performance with that of CVH-REC.

Fig.11 illustrates the performance of CVH-REC$^{\text{-semantic}}$, and we can observe from the figure that assigning the semantic vectors to entities positively affects the web API recommendation on Precision, Recall, MAP, and NDCG metrics. Unlike random vectors, the semantic vectors of entities inherently encode domain-specific knowledge. By adjusting these semantic vectors to form the initial entity vectors, HGNNs can leverage this prior knowledge during the message-passing process. This allows the learned final entity vectors to not only capture the structured knowledge of hypergraphs but also retain domain-specific knowledge.

4.10 Threats to Validity

Threats to construct validity. In our work, a potential construct threat arose from the correctness of the evaluation metrics. To mitigate this risk, we employed standard metrics—Precision, Recall, MAP, and NDCG. These metrics have been extensively used in prior studies on web API recommendation.

Another potential threat to construct validity stems from the choice of statistical metric: specifically, we adopted the mean rather than the median of the evaluation results across all mashup requirements. Compared to the median (a robust measure of central tendency that is insensitive to extreme values), the mean is more sensitive to the performance differences across all test cases—especially the improvements in the scenarios where the baseline methods underperform. This characteristic makes the mean a more suitable metric for quantifying the overall performance gains of CVH-REC over the baseline methods.

Threats to internal validity. A potential threat arose from the way we evaluated the performance of recom-

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

mendation methods, which involved comparing the recommended web APIs with those actually used in mashups. However, since mashups can often be created using different combinations of web APIs, some reasonable recommendations made by methods might have been incorrectly considered inappropriate.

A small dataset may impair the reliability of the experimental results. However, nearly all existing studies on web API recommendation (e.g., [1], [4], [31]) construct their experimental datasets using 3,000 to 8,000 mashups, while we employ 7,163 mashups to evaluate the performance of the proposed method. Therefore, we believe the impact of this threat on our experiment is limited. In the future, we will expand the experimental dataset by collecting data from different platforms or collaborating with mashup developers, thereby better evaluating the recommendation method.

CVH-REC has relatively high model complexity compared to experimental dataset size, potentially leading to overfitting and undermining the reliability of experimental results. To address this, we calculated the gaps in various evaluation metrics of CVH-REC between the training set and the test set. The results show that under different metrics, the training set performance is only 6%-10% higher than that of the test set (absolute gap). This indicates that CVH-REC does not suffer from severe overfitting and has stable generalization ability. Therefore, the actual impact of this potential overfitting risk is limited.

For internal validity, insufficient statistical testing and run variability may pose key threats. To mitigate these, we applied Bonferroni-corrected p-values for rigorous statistical testing and calculated Cohen's d values to quantify effect sizes, alongside adopting 10-fold cross-validation with 10 repeated experiments (reporting standard deviations). These measures ensure findings are statistically robust, meaningful, and reproducible, thereby enhancing internal validity.

Threats to external validity. A potential threat was related to the selection of baseline methods. Using a narrow or outdated set of baseline methods might have overstated the advantages of the proposed method. To mitigate this risk, we included a variety of baseline methods covering both traditional and recent state-of-the-art methods. Specifically, we adopted the popularity-based method POP, which is commonly used as a basic baseline in many studies, and also selected recent methods published in high-impact conferences and journals. These representative baselines ensured a fair and meaningful comparison.

5 Related Work

This section introduces works related to web API recommendation, including content-based, CF-based, and deep learning-based methods.

5.1 Content-Based Methods

Content-based methods assist developers in discovering suitable web APIs by analyzing the semantic relationships between requirement texts and the textual descriptions of web APIs. Li et al. [32] employ probabilistic modeling to uncover latent topic structures in requirement documents and web API descriptions, enabling semantic similarity computation and estimation of mashup–API matching. Shi et al. [9] present a tag-attention Siamese BiLSTM model for web API recommendation that jointly embeds web APIs and mashup requirements, using tags of web API to guide attention to functionally relevant words. Zhao et al. [33] combine multiple types of features including textual, nearest neighbor, API-specific, and tag features to estimate the relevance between mashup requirements and web APIs using a regression model. Zhong et al. [34] generate comprehensive web API representations from usage history to bridge the vocabulary gap between mashup developers and web API providers. By leveraging the Author-Topic Model to capture web API-word co-occurrences, these representations can be dynamically refined and enriched. Sang et al. [35] propose SAMF, a model that combines Bayesian matrix factorization with Doc2Vec to mine the semantics of mashup and web API descriptions. Text similarity and adversarial training are incorporated to alleviate data sparsity and enhance recommendation accuracy and robustness. However, the performance of content-based methods is often hindered by the semantic gap between requirement texts and web API descriptions, including both the vocabulary gap and the intention gap. This makes it difficult for these methods to accurately align developer needs with appropriate web APIs.

5.2 Collaborative Filtering-Based Methods

Collaborative filtering (CF) has been extensively utilized in web API recommendation tasks [36], [37]. CF-based methods recommend web APIs to a mashup requirement by identifying its similar mashups from historical data and generating the list of web APIs based on the web APIs employed by these mashups. Rahman et al. [38] introduce a user-based collaborative filtering method for web API recommendation, which leverages a two-level topic model to discover mashups that are similar to the requirement. Xiong et al. [39] combine collaborative filtering and textual content to integrate the invocation interactions between mashups and web APIs and their functional descriptions through neural networks. Yao et al. [40], [41] propose a probabilistic matrix factorization model enhanced with implicit correlation regularization, aiming to improve the quality of web API recommendation while promoting greater diversity in the recommended results. Fletcher et al. [42] incorporate the implicit user preferences (i.e., invocation history) into the matrix factorization model to improve the accuracy and diversity of recommendations. Despite the remarkable success achieved by many CF-based methods, they are often susceptible to the popularity bias.

5.3 Deep Learning-Based Methods

With the advancement of deep learning across various domains, many recent studies adopt them to address the problem of web API recommendation. Fan et al. [27] present GSR, a seq2seq-based web API recommendation model that leverages reinforcement learning to handle sequence dependency. Xiao et al. [43] propose a novel group feature aggregation web API recommendation framework that employs a dual-attention mechanism to capture effective

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted,
but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

feature and preference information of group members for each user, thereby enhancing feature representation. Kang et al. [44] propose a neural factorization machine enhanced with domain interactions and self-attention for web API recommendation, where domain interactions group related features and self-attention captures the importance of their interactions. Yu et al. [45] introduce an iterative web API recommendation framework that fuses features from requirements and interaction history and uses a tag judgment task to enhance prediction of the next web API.

Recent studies have increasingly adopted GNNs for web API recommendation to model complex relationships [31], [46], [47]. Zheng et al. [48] introduce a novel positional and semantic encoding method and motif-based linearizing graph Transformer for automatic web API recommendation. Cao et al. [49] utilize a heterogeneous graph attention network to learn semantic representations of web APIs from meta-paths and attributes, and integrates them with multi-dimensional QoS features to capture feature importance and interactions. Kang et al. [7] propose CLJT, which integrates BERT and LightGCN to capture textual and structural semantics, and employs contrastive learning to align them for improved recommendation performance. We propose R2API [15], an HGNN-based web API recommendation method that builds four homogeneous hypergraphs and employs multi-task learning to obtain entity vectors. However, it has a key limitation: using simple homogeneous hypergraphs to describe mashup or web API entities fails to comprehensively and accurately characterize them, undermining the quality of learned entity vectors.

6 Conclusion and Future work

This work proposes a novel web API recommendation method, i.e., CVH-REC, which involves three main steps. First, CVH-REC models the interactions between mashups and web APIs, as well as their tag usage records, as a multi-view knowledge graph to comprehensively and accurately characterize entities. Second, CVH-REC adopts a contrastive learning and multi-task learning framework to drive multiple HGNNs in jointly learning entity vectors. Third, CVH-REC obtains the semantic vector of the mashup requirement using the SBERT model and transfers it into the vector space of the knowledge graph with an MLP. CVH-REC generates a recommendation list of web APIs by comparing the vector of the mashup requirement with those of web APIs. Through a series of research questions and experiments, we verified that CVH-REC performs better than the baseline methods.

In future work, we aim to generate explanatory texts for web API recommendation lists, thereby helping software developers better understand and utilize the recommended web APIs. The robust text generation capabilities of LLMs hold promise for achieving this objective.

Acknowledgments

The work is funded by the Science Research Project of Hebei Education Department (QN2026548).

References

[1] Y. Wang, A. Zhou, Q. Huang, X. Wang, and B. Jiang, "Parei: A progressive approach for web api recommendation by combining explicit and implicit information," *Information and Software Technology*, vol. 162, p. 107269, 2023.

[2] W. Gong, X. Zhang, Y. Chen, Q. He, A. Beheshti, X. Xu, C. Yan, and L. Qi, "Dawar: Diversity-aware web apis recommendation for mashup creation based on correlation graph," in *Proceedings of the 45th international ACM SIGIR conference on Research and Development in information retrieval*, 2022, pp. 395–404.

[3] B. Cao, X. F. Liu, J. Liu, and M. Tang, "Domain-aware mashup service clustering based on lda topic model from multiple data sources," *Information and Software Technology*, vol. 90, pp. 40–54, 2017.

[4] X. Hu, "Biased random walk based web api recommendation in heterogeneous network," in *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024, pp. 172–177.

[5] G. Zou, L. Wu, S. Hu, S. Yang, C. Zhou, Y. Gan, and B. Zhang, "User profiling for personalized service recommendation with dual high-order feature learning," in *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024, pp. 269–280.

[6] Y. Ma, X. Geng, and J. Wang, "A deep neural network with multiplex interactions for cold-start service recommendation," *IEEE Transactions on Engineering Management*, vol. 68, no. 1, pp. 105–119, 2020.

[7] G. Kang, H. Ren, W. Chen, J. Liu, B. Cao, and Y. Xu, "Web api recommendation via exploring textual and structural semantics with contrastive learning and joint training," *IEEE Transactions on Network and Service Management*, 2025.

[8] H. Wu, Y. Duan, K. Yue, and L. Zhang, "Mashup-oriented web api recommendation via multi-model fusion and multi-task learning," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3330–3343, 2022.

[9] M. Shi, Y. Tang, and J. Liu, "Ta-blstm: tag attention-based bidirectional long short-term memory for service recommendation in mashup creation," in *2019 international joint conference on neural networks (IJCNN)*. IEEE, 2019, pp. 1–8.

[10] Q. He, B. Li, F. Chen, J. Grundy, X. Xia, and Y. Yang, "Diversified third-party library prediction for mobile app development," *IEEE Transactions on Software Engineering*, vol. 48, no. 1, pp. 150–165, 2020.

[11] P. T. Nguyen, R. Rubei, J. Di Rocco, C. Di Sipio, D. Di Ruscio, and M. Di Penta, "Dealing with popularity bias in recommender systems for third-party libraries: How far are we?" in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 2023, pp. 12–24.

[12] C. Wei, Y. Fan, and J. Zhang, "Time-aware service recommendation with social-powered graph hierarchical attention network," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 2229–2240, 2023.

[13] X. Zheng, G. Wang, J. Zhang, Y. Zhang, N. Wang, J. Yu, and Y. Han, "H-mgsr: a hierarchical motif-based graph attention neural network for service recommendation," in *2023 IEEE international conference on web services (ICWS)*. IEEE, 2023, pp. 553–562.

[14] Z. Zhang, Y. Zhang, M. Dong, K. Ota, Y. Zhang, and Y. Ren, "Collaborative tag-aware graph neural network for long-tail service recommendation," *IEEE Transactions on Services Computing*, vol. 17, no. 5, pp. 2124–2138, 2024.

[15] Y. Wang, X. Kang, X. Li, and S. Gao, "R2api: A novel method for web api recommendation by using hgnns with multi-task learning," *IEEE Transactions on Software Engineering*, 2025.

[16] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992. [Online]. Available: https://aclanthology.org/D19-1410/

[17] M. Riedmiller and A. Lernen, "Multi layer perceptron," *Machine learning lab special lecture, University of Freiburg*, vol. 24, pp. 11–60, 2014.

[18] S. Ji, Y. Feng, R. Ji, X. Zhao, W. Tang, and Y. Gao, "Dual channel hypergraph collaborative filtering," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 2020–2029.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3666184

[19] L. Chen, N. Mei, Y. He, W. Liu, G. Zhong, and M. Tang, "High-order collaborative filtering for third-party library recommendation," in *2023 IEEE International Conference on Web Services (ICWS)*. IEEE, 2023, pp. 483–492.

[20] M. Jing, Y. Zhu, T. Zang, and K. Wang, "Contrastive self-supervised learning in recommender systems: A survey," *ACM Transactions on Information Systems*, vol. 42, no. 2, pp. 1–39, 2023.

[21] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020.

[22] Y. Zhang and Q. Yang, "A survey on multi-task learning," *IEEE transactions on knowledge and data engineering*, vol. 34, no. 12, pp. 5586–5609, 2021.

[23] ——, "An overview of multi-task learning," *National Science Review*, vol. 5, no. 1, pp. 30–43, 2018.

[24] M. Liu, Z. Tu, Y. Zhu, X. Xu, Z. Wang, and Q. Z. Sheng, "Data correction and evolution analysis of the programmableweb service ecosystem," *Journal of Systems and Software*, p. 111066, 2021.

[25] T. Yu, L. Zhang, H. Liu, H. Liu, and J. Wang, "Service recommendation based on contrastive learning and multi-task learning," *Computer Communications*, vol. 213, pp. 285–295, 2024.

[26] S. Liu, M. Liu, T. Jiang, S. Yu, H. Xu, and Z. Wang, "Gsl-mash: Enhancing mashup creation service recommendations through graph structure learning," in *International Conference on Service-Oriented Computing*. Springer, 2024, pp. 176–191.

[27] G. Fan, S. Chen, Q. He, H. Wu, J. Li, X. Xue, and Z. Feng, "Service recommendations for mashup based on generation model," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1820–1834, 2024.

[28] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "The impact of automated parameter optimization on defect prediction models," *IEEE Transactions on Software Engineering*, vol. 45, no. 7, pp. 683–711, 2018.

[29] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[30] F. Kazemi, M. Lamothe, and S. McIntosh, "Exploring the notion of risk in code reviewer recommendation," in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2022, pp. 139–150.

[31] X. Wang, M. Xi, Y. Li, X. Pan, Y. Wu, S. Deng, and J. Yin, "Sehgn: Semantic-enhanced heterogeneous graph network for web api recommendation," *IEEE Transactions on Services Computing*, 2024.

[32] C. Li, R. Zhang, J. Huai, X. Guo, and H. Sun, "A probabilistic approach for web service discovery," in *2013 ieee international conference on services computing*. IEEE, 2013, pp. 49–56.

[33] H. Zhao, J. Wang, Q. Zhou, X. Wang, and H. Wu, "Web api recommendation with features ensemble and learning-to-rank," in *CCF Conference on Big Data*. Springer, 2019, pp. 406–419.

[34] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 2, pp. 468–478, 2018.

[35] C. Sang, X. Deng, and S. Liao, "Mashup-oriented web api recommendation via full-text semantic mining of developer requirements," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2755–2768, 2023.

[36] Y. Hu, Q. Peng, X. Hu, and R. Yang, "Time aware and data sparsity tolerant web service recommendation based on improved collaborative filtering," *IEEE Transactions on Services Computing*, vol. 8, no. 5, pp. 782–794, 2014.

[37] B. Cao, X. F. Liu, M. M. Rahman, B. Li, J. Liu, and M. Tang, "Integrated content and network-based service clustering and web apis recommendation for mashup development," *IEEE Transactions on Services Computing*, vol. 13, no. 1, pp. 99–113, 2020.

[38] M. M. Rahman, X. Liu, and B. Cao, "Web api recommendation for mashup development using matrix factorization on integrated content and network-based service clustering," in *2017 IEEE international conference on services computing (SCC)*. IEEE, 2017, pp. 225–232.

[39] R. Xiong, J. Wang, N. Zhang, and Y. Ma, "Deep hybrid collaborative filtering for web service recommendation," *Expert systems with Applications*, vol. 110, pp. 191–205, 2018.

[40] L. Yao, X. Wang, Q. Z. Sheng, W. Ruan, and W. Zhang, "Service recommendation for mashup composition with implicit correlation regularization," in *2015 IEEE International Conference on Web Services*. IEEE, 2015, pp. 217–224.

[41] L. Yao, X. Wang, Q. Z. Sheng, B. Benatallah, and C. Huang, "Mashup recommendation by regularizing matrix factorization with api co-invocations," *IEEE Transactions on Services Computing*, vol. 14, no. 2, pp. 502–515, 2018.

[42] K. Fletcher, "Regularizing matrix factorization with implicit user preference embeddings for web api recommendation," in *2019 IEEE international conference on services computing (SCC)*. IEEE, 2019, pp. 1–8.

[43] Y. Xiao, J. Liu, G. Kang, and B. Cao, "Group feature aggregation for web service recommendations," *IEEE Transactions on Network and Service Management*, 2024.

[44] G. Kang, L. Ding, J. Liu, B. Cao, and Y. Xu, "Web api recommendation based on self-attentional neural factorization machines with domain interactions," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 6, pp. 3953–3963, 2023.

[45] T. Yu, D. Yu, D. Wang, Q. Yang, and X. Hu, "Iterative framework based on multi-task learning for service recommendation," *Journal of Systems and Software*, vol. 207, p. 111873, 2024.

[46] B. Cao, Q. Peng, X. Xie, Z. Peng, J. Liu, and Z. Zheng, "Web service recommendation via combining topic-aware heterogeneous graph representation and interactive semantic enhancement," *IEEE Transactions on Services Computing*, 2024.

[47] G. Kang, Y. Wang, H. Ren, B. Cao, J. Liu, and Y. Wen, "Ks-gnn: Keyword search via graph neural network for web api recommendation," *IEEE Transactions on Network and Service Management*, 2024.

[48] X. Zheng, G. Wang, Y. Zhang, B. Han, and J. Yu, "Motif-based linearizing graph transformer for web api recommendation," in *International Conference on Service-Oriented Computing*. Springer, 2024, pp. 138–145.

[49] B. Cao, M. Peng, L. Zhang, Y. Qing, B. Tang, G. Kang, and J. Liu, "Web service recommendation via integrating heterogeneous graph attention network representation and fibinet score prediction," *IEEE Transactions on Services Computing*, vol. 16, no. 5, pp. 3837–3850, 2023.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on February 20,2026 at 17:59:20 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.